

Formation avancée en Deep Learning avec PyTorch

Computer Vision & Séries temporelles

Plan de la formation

Niveau Intermédiaire / Avancé



Python 3.10+

NumPy, Pandas



PyTorch 2.x

torchvision



Computer Vision

CNN, ViT, GAN



Séries temporelles

RNN, LSTM, GRU

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

ENETCOM — Université de Sfax

Présentation 00 — Plan de la formation

1. Présentation générale
2. Vue d'ensemble des modules
3. Module 1 — Initiation PyTorch, MLP & CNN
4. Module 2 — Transfer Learning, Séquences & XAI
5. Module 3 — Autoencoders, VAE & Diffusion
6. Module 4 — GAN, ViT & Projet
7. Stack technique & ressources

Présentation générale

À propos de cette formation

Objectif global

Maîtriser les architectures fondamentales *et* avancées du **Deep Learning** appliquées à la **Computer Vision**, entièrement avec **PyTorch**.

- Du **MLP** aux **Vision Transformers** (ViT, Swin)
- **RNN**, **LSTM**, **GRU** pour séries temporelles
- Modèles génératifs : **VAE**, **GAN**, **Diffusion**
- Interprétabilité : **Grad-CAM**, **SHAP**, **LIME**
- Pipeline complet : data → entraînement → déploiement

Informations clés



Intermédiaire / Avancé



Présentiel



12 à 25 participants



Python, PyTorch
torchvision, OpenCV, HF



Projet fil rouge évalué

À l'issue de la formation, les participants seront capables de :

- **Maîtriser PyTorch** : tenseurs, autograd, `nn.Module`, `DataLoader`
- Implémenter MLP, CNN, RNN/LSTM, AE/VAE, GAN, ViT
- **Computer Vision** : classification, détection, segmentation, débruitage
- **Séries temporelles** : classification, forecasting, anomalies
- Concevoir des **modèles génératifs** (VAE, GAN, Diffusion)
- Exploiter **Transfer Learning** (`torchvision`, `timm`, HF)
- Déployer un modèle : `TorchScript`, `ONNX`, quantification
- Appliquer l'**Explainable AI** : Grad-CAM, SHAP, LIME
- Implémenter les **modèles de diffusion** (DDPM)

✓ Prérequis

- Bases solides en **Python** (NumPy, Pandas, Matplotlib)
- Notions de Machine Learning classique
- Maths : algèbre linéaire, calcul diff., probabilités
- Aucune connaissance préalable de PyTorch requise

👤 Public cible

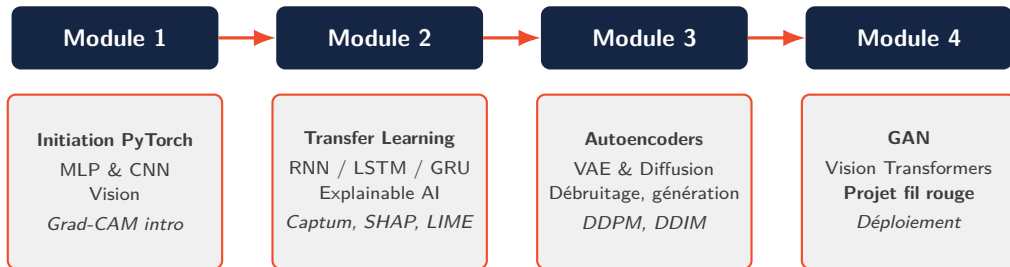
- Chercheurs en vision par ordinateur
- Ingénieurs ML / Data Scientists
- Doctorants et étudiants avancés en IA
- Praticiens en imagerie médicale et industrielle

Vue d'ensemble des modules

Cartographie — architectures & cas d'usage par module

Mod.	Thème	Architectures clés	Cas d'usage
M1	Fondations PyTorch	Tenseurs, autograd, nn.Module , DataLoader	Pipeline data → train → éval
M1	MLP & bases DL	Perceptron, couches denses, BatchNorm, Dropout	Classification (Fashion-MNIST)
M1	CNN & vision	LeNet, VGG, ResNet (skip), EfficientNet	Classification d'images, backbones
M2	Transfer Learning	torchvision , timm , fine-tuning, OneCycleLR	Classif. médicale & industrielle
M2	Séquences (RNN)	RNN, LSTM , GRU, BiLSTM	Forecasting, anomalies (séries temp.)
M2	Explainable AI	Grad-CAM, Integrated Grad., SHAP, LIME	Audit, debug, conformité
M3	Autoencoders	AE, ConvAE, Denoising AE, U-Net	Débruitage, compression, anomalies
M3	VAE	VAE, β -VAE, Conditional VAE	Génération, interpolation latente
M3	Diffusion	DDPM, DDIM, score-based, CFG	Génération, inpainting, super-rés.
M4	GAN	DCGAN, WGAN-GP, cGAN, Pix2Pix, SRGAN	Super-résolution, traduction d'image
M4	Vision Transformers	ViT , DeiT, Swin, SegFormer, DINO	Classif. fine, segmentation
M4	Déploiement	TorchScript, ONNX, quantification int8	Mise en production, edge / mobile

Progression pédagogique — 4 modules



Fondations → Interprétabilité → Génération → Déploiement

Module 1 — Initiation PyTorch, MLP & CNN

Module 1 — Initiation PyTorch, MLP & CNN

🎯 Objectifs

Maîtriser les fondamentaux de PyTorch et implémenter MLP et CNN pour la Computer Vision.

Session	Contenu & activités
Initiation PyTorch	Tenseurs vs NumPy, GPU/CPU, autograd, graphe de calcul, <code>nn.Module</code> , optimiseurs
TP — Premiers pas	Dataset/DataLoader custom, boucle train/val/test, <code>state_dict</code> , checkpoints
MLP & fondamentaux DL	Activations (ReLU, GELU), forward/backprop, SGD-Momentum/Adam, Dropout, BatchNorm
TP MLP — Images	Classification Fashion-MNIST avec <code>nn.Sequential</code> , TensorBoard / W&B
CNN — théorie	<code>Conv2d</code> , padding/stride, pooling, <i>receptive field</i> , ResNet & skip connections , EfficientNet
TP CNN — Vision	<code>torchvision.datasets</code> , transforms/augmentation, matrice de confusion, ROC/AUC (<code>torchmetrics</code>)

Module 2 — Transfer Learning, Séquences & XAI

Module 2 — Transfer Learning, Séquences & Explainability

🎯 Objectifs

Exploiter le Transfer Learning, maîtriser les architectures récurrentes (RNN, LSTM, GRU) pour les séries temporelles, et appliquer les techniques d'Explainable AI.

Session	Contenu & activités
Transfer Learning	torchvision.models , anatomie VGG16, gel/dégel, têtes custom, timm , OneCycleLR
TP Fine-Tuning	ResNet50 sur Chest X-Ray, EfficientNet (défauts industriels), domain adaptation
RNN, LSTM & GRU	BPTT, vanishing gradient, 4 portes LSTM, GRU, BiLSTM, pack_padded_sequence
TP Séries temporelles	Forecasting (énergie, IoT), classification nn.LSTM , détection d'anomalies AE+LSTM
Explainable AI (XAI)	Grad-CAM, Integrated Gradients, SHAP, LIME, Attention Rollout (ViT)
TP XAI	Grad-CAM sur CNN médical (Captum), LIME sur images, comparaison des méthodes

Module 3 — Autoencoders, VAE & Diffusion

Module 3 — Autoencoders, VAE & Modèles de Diffusion

🎯 Objectifs

Maîtriser AE/ConvAE et VAE pour débruitage, compression et détection d'anomalies. Implémenter les modèles de diffusion (DDPM) pour la génération d'images.

Session	Contenu & activités
Autoencoders	Encodeur/décodeur, MSE, ConvTranspose2d , Denoising AE, U-Net
TP AE — débruitage	ConvAE sur IRM/CT, anomalies par erreur de reconstruction, latent + UMAP
VAE	Reparametrization trick, KL divergence, ELBO, β -VAE, Conditional VAE
TP VAE	Génération MNIST/CelebA, interpolation latente, conditionnement par classe
Diffusion — DDPM	Forward/reverse process, noise schedule (linéaire/cosinus), U-Net + timestep, vue score-based
TP Diffusion	Training loop DDPM, DDIM sampling, classifier-free guidance, diffusers (HF)

Module 4 — GAN, ViT & Projet

Module 4 — GAN, Vision Transformers & Projet fil rouge

🎯 Objectifs

Maîtriser les architectures GAN, les Vision Transformers (ViT, Swin, SegFormer) et réaliser un **projet fil rouge** intégrant l'ensemble des compétences.

Session	Contenu & activités
GAN — théorie	Jeu minimax, DCGAN, cGAN, Pix2Pix (PatchGAN), WGAN-GP , CycleGAN
TP GAN	DCGAN MNIST/CelebA, SRGAN super-résolution, métriques FID/IS/SSIM/PSNR
Vision Transformers	Self-attention, patch embedding, ViT, DeiT, Swin, SegFormer (timm , HF)
TP ViT — Segmentation	Fine-tuning ViT/Swin, SegFormer, attention maps, CNN vs ViT
Déploiement	TorchScript (trace / script), ONNX, quantification int8, PyTorch Mobile
Projet fil rouge	Pipeline complet sur use case réel : critères, livrables & démo

Projet fil rouge — thèmes proposés



Thèmes proposés (archi · métriques)

- Imagerie médicale : classif./segmentation (IRM, rétinopathie) — [ResNet/ViT](#), [U-Net/SegFormer](#) · F1, mIoU, Dice
- Contrôle qualité industriel : défauts + localisation — [CNN](#) + [Grad-CAM](#) · F1, mAP
- Maintenance prédictive : RUL (NASA C-MAPSS) — [LSTM](#) + [CNN 1D](#) · MAE, RMSE
- Super-résolution satellite (Sentinel-2) — [SRGAN](#) / Diffusion · PSNR, SSIM, FID
- Segmentation urbaine (CityScapes) — [SegFormer](#) · mIoU
- Génération médicale par diffusion (IRM/CT) — [DDPM](#) conditionnel · FID + F1 aval
- Audit XAI d'un classifieur médical — [Grad-CAM](#) + [SHAP](#) · fidélité



Critères d'évaluation

- Pertinence du choix architectural et justification
- Qualité du pipeline (prétraitement, augmentation, validation croisée)
- Performance (*accuracy*, *mAP*, *IoU*, *SSIM*...)
- Reproductibilité (seeds, logging W&B / TensorBoard)
- Clarté de la présentation et démo *live*

Stack technique & ressources

⌘ Cœur du framework

- [Python 3.10+](#) (Conda / venv)
- [PyTorch 2.x](#) — [torchvision](#)
- [timm](#) — zoo ViT, Swin, EfficientNet
- Hugging Face [transformers](#), [diffusers](#)
- [torchmetrics](#), [einops](#), OpenCV

🔍 XAI & MLOps

- [Captum](#) & [pytorch-grad-cam](#)
- [pytorch-fid](#), SHAP, LIME
- TensorBoard / Weights & Biases
- Déploiement : [TorchScript](#), [ONNX](#)
- GPU : NVIDIA CUDA 11.8+, Colab Pro

TP reproductibles sur Google Colab (GPU T4) ou poste local CUDA.

Ressources — ouvrages de référence

Ouvrage	Auteur(s)	Éditeur / Année
Fundamentals of Deep Learning (2 ^e éd.) Foundations & neural networks	Nithin Buduma, Nikhil Buduma, Joe Papa	O'Reilly, 2022
Python Deep Learning (2 ^e éd.) PyTorch, Keras, TensorFlow	I. Vasilev, D. Slater, G. Spacagna, P. Roelants, V. Zocca	Packt, 2019
Advanced Deep Learning with Keras Autoencoders, GANs, VAEs, RL	Rowel Atienza	Packt, 2018

Les ouvrages complets sont disponibles dans le dossier [md_books/](#) du repository de la formation.

Prêts à plonger dans le Deep Learning ?

Cap sur le Module 1 — PyTorch from scratch

✉ bassem.benhamed@enetcom.usf.tn